


Lecture 7: Bridging optimisation & learning: unrolling and plug & play approaches

---

Luca Calatroni
before: CR CNRS, Laboratoire I3S
now: MaLGa, University of Genoa, Italy

MSc DSAI - UCA

Inverse problems in biological imaging

February 25 2025

Unrolled algorithms

Back to PGD, FB, ISTA

$$\min_{x \in \mathbb{R}^n} \ \frac{1}{2}\|Ax - y\|^2 + \lambda \|x\|_1$$

[1] Daubechies, Defrise, De Mol, 2004

Back to PGD, FB, ISTA

$$\min_{x \in \mathbb{R}^n} \frac{1}{2}\|Ax - y\|^2 + \lambda \|x\|_1$$

Iterative Soft Thresholding Algorithm (ISTA)[1]

The PGD iterations take the form: for $x_0 \in \mathbb{R}^n$, $\tau \in (0, 2/L)$ and $k \geq 0$

$$x_{k+1} = \text{prox}_{\tau\lambda\|\cdot\|_1}(x_k - \tau A^T(Ax_k - y)) = \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y))$$

where $\mathcal{T}_{\tau\lambda}(z)$ is the *soft-thresholding* (or *shrinkage*) operator (prox of the multivariate function $g(x) = \lambda\|x\|_1$ with parameter τ) defined by:

$$\mathcal{T}_{\tau\lambda}(z) = (\mathcal{T}_{\tau\lambda}(z_j))_{j=1,\dots,n} = \Big(\big[|z_j| - \lambda\tau\big]_+ \text{sign}(z_j) \Big)_{j=1,\dots,n}$$

[1] Daubechies, Defrise, De Mol, 2004

Back to PGD, FB, ISTA

$$\min_{x \in \mathbb{R}^n} \frac{1}{2}\|Ax - y\|^2 + \lambda \|x\|_1$$

Iterative Soft Thresholding Algorithm (ISTA)[1]

The PGD iterations take the form: for $x_0 \in \mathbb{R}^n$, $\tau \in (0, 2/L)$ and $k \geq 0$

$$x_{k+1} = \text{prox}_{\tau\lambda\|\cdot\|_1}(x_k - \tau A^T(Ax_k - y)) = \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y))$$

where $\mathcal{T}_{\tau\lambda}(z)$ is the *soft-thresholding* (or *shrinkage*) operator (prox of the multivariate function $g(x) = \lambda\|x\|_1$ with parameter τ) defined by:

$$\mathcal{T}_{\tau\lambda}(z) = (\mathcal{T}_{\tau\lambda}(z_j))_{j=1,\dots,n} = \Big(\big[|z_j| - \lambda\tau\big]_+ \text{sign}(z_j) \Big)_{j=1,\dots,n}$$

- In practice: iterate **till convergence** (check relative error $\|x_{k+1} - x_k\|$ or $F(x_{k+1}) - F(x_k) \leq \epsilon_{\text{tol}}$)

[1] Daubechies, Defrise, De Mol, 2004

Back to PGD, FB, ISTA

$$\min_{x \in \mathbb{R}^n} \frac{1}{2}\|Ax - y\|^2 + \lambda \|x\|_1$$

Iterative Soft Thresholding Algorithm (ISTA)[1]

The PGD iterations take the form: for $x_0 \in \mathbb{R}^n$, $\tau \in (0, 2/L)$ and $k = 0, \dots, K-1$ iteration

$$x_{k+1} = \text{prox}_{\tau\lambda\|\cdot\|_1}(x_k - \tau A^T(Ax_k - y)) = \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y))$$

where $\mathcal{T}_{\tau\lambda}(z)$ is the *soft-thresholding* (or *shrinkage*) operator (prox of the multivariate function $g(x) = \lambda\|x\|_1$ with parameter τ) defined by:

$$\mathcal{T}_{\tau\lambda}(z) = (\mathcal{T}_{\tau\lambda}(z_j))_{j=1,\dots,n} = \Big(\big[|z_j| - \lambda\tau\big]_+ \text{sign}(z_j) \Big)_{j=1,\dots,n}$$

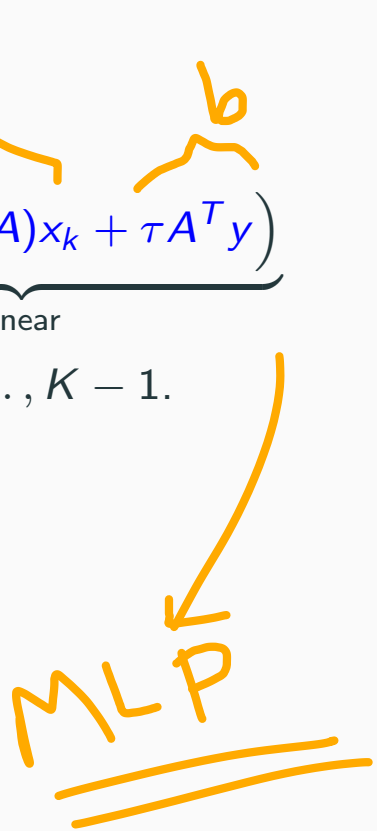
- In practice: iterate **till convergence** (check relative error $\|x_{k+1} - x_k\|$ or $F(x_{k+1}) - F(x_k) \leq \epsilon_{\text{tol}}$)
- What if we stop at a certain K? (max. computational budget)

[1] Daubechies, Defrise, De Mol, 2004

A neural network interpretation of ISTA

Look at one iteration step:

$$x_{k+1} = \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y)) = \underbrace{\mathcal{T}_{\tau\lambda}}_{\approx \text{ReLU}} \underbrace{\left((\mathrm{Id} - \tau A^T A)x_k + \tau A^T y\right)}_{\text{linear}}$$



. . . clear analogy with k-th layer of a network, at each $k = 0, \ldots, K-1$.

[2] Gregor, LeCun, '10

A neural network interpretation of ISTA

Look at one iteration step:

$$x_{k+1} = \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y)) = \underbrace{\mathcal{T}_{\tau\lambda}}_{\approx \text{ReLU}} \underbrace{\left((\text{Id} - \tau A^T A)x_k + \tau A^T y\right)}_{\text{linear}}$$

. . . clear analogy with k-th layer of a network, at each $k = 0, \ldots, K-1$.

Interpretation: K-layer network obtained by unrolling/unfolding the iterations[2]:

$$\begin{aligned} x_{k+1} &= \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y)) \\ &= \mathcal{T}_{\tau\lambda}\left(\mathcal{T}_{\tau\lambda}\left(x_{k-1} - \tau A^T(Ax_{k-1} - y)\right) - \tau A^T\left(A\left(\mathcal{T}_{\tau\lambda}\left(x_{k-1} - \tau A^T(Ax_{k-1} - y)\right) - y\right)\right)\right) \\ &= \ldots \\ &= \mathcal{T}_{\tau\lambda}\left(\mathcal{T}_{\tau\lambda}\left(\ldots\left(x_0 - \tau A^T(Ax_0 - y)\right)\ldots\right)\right) \end{aligned}$$

Same parameters $\{A, \tau, \lambda\}$ shared across the network (recurrent NN).

[2] Gregor, LeCun, '10

The ISTA "network"

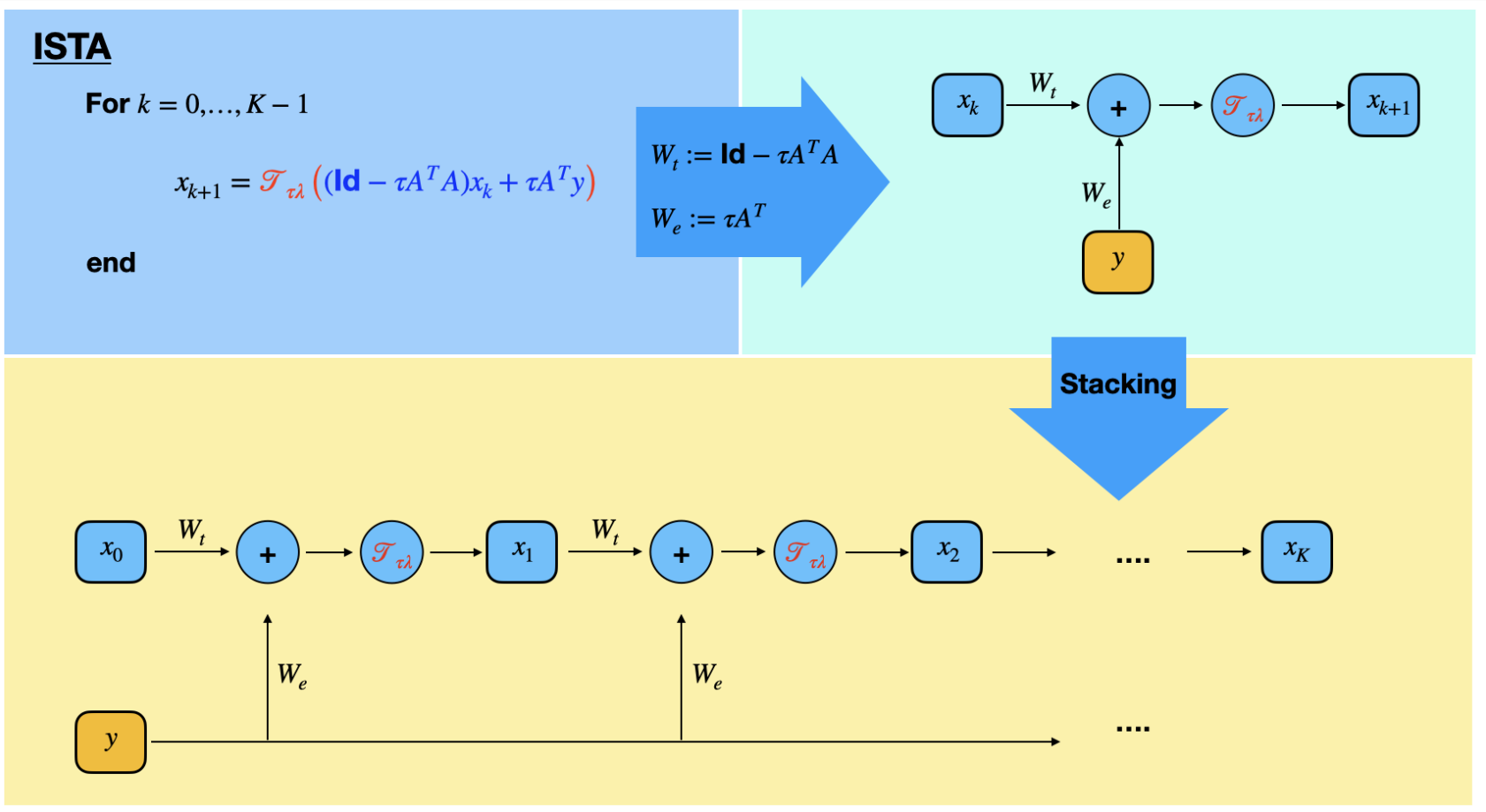


Figure 1: Parameters of the network are: $\{W_t, W_e, \tau\}$ (potentially, also λ).

Learned ISTA (LISTA) & generalizations

- Given a dataset $(\tilde{x}_i, y_i)$, $i = 1, \dots S$, learn $\{W_t, W_e, \tau\}$ by minimising:

$$\mathcal{L}(W_t, W_e, \tau) := \frac{1}{2S} \sum_{i=1}^{S} \|x_i^{K-1}(W_t, W_e, \tau; y_i) - \tilde{x}_i\|^2$$

- Use SGD (or variants) for training
- Notice that $x_i^{K-1}(W_t, W_e, \tau; y_i)$ is computed in a fixed number of K iterations.
- It was observed[3] that, after training, the number of layers K required to get convergence on a new output is smaller than the one required by PGD.

[3] Gregor, LeCun, '10

Learned ISTA (LISTA) & generalizations

- Given a dataset $(\tilde{x}_i, y_i)$, $i = 1, \dots S$, learn $\{W_t, W_e, \tau\}$ by minimising:

$$\mathcal{L}(W_t, W_e, \tau) := \frac{1}{2S} \sum_{i=1}^{S} \|x_i^{K-1}(W_t, W_e, \tau; y_i) - \tilde{x}_i\|^2$$

- Use SGD (or variants) for training
- Notice that $x_i^{K-1}(W_t, W_e, \tau; y_i)$ is computed in a fixed number of K iterations.
- It was observed[3] that, after training, the number of layers K required to get convergence on a new output is smaller than the one required by PGD.

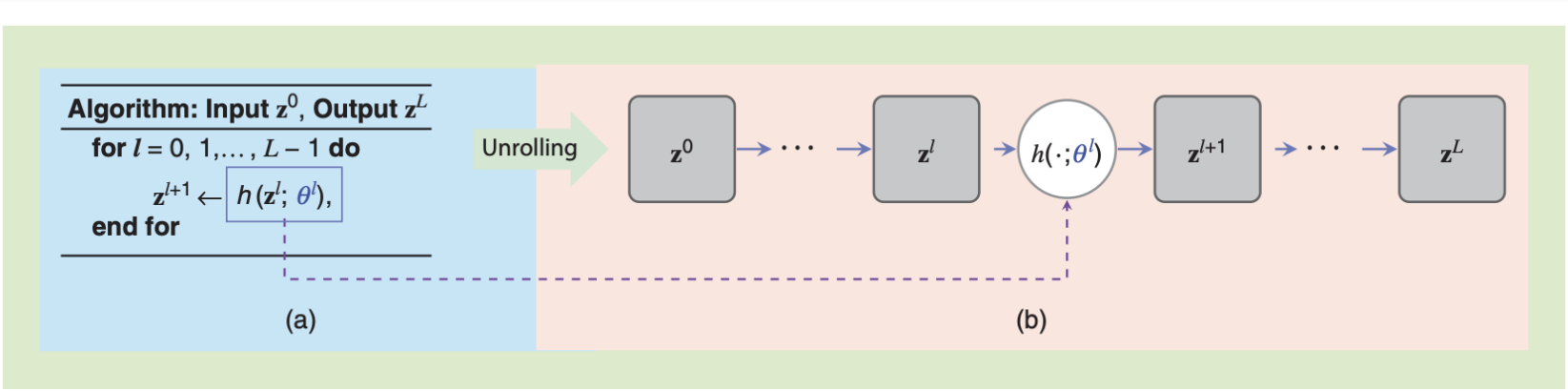


Figure 2: General structure of algorithmic unrolling.

[3] Gregor, LeCun, '10

Actual unrolling

Let $\left\{W_t^k, W_e^k, \tau^k, \lambda^k\right\}_{k=0}^{K-1}$ vary at each $k = 0, \dots, K-1$ [4].

Similar to NN's: weights/bias vectors + non-linearity (encoded in τ^k) are learned end-to-end.

$$\mathcal{L}(\left\{W_t^k, W_e^k, \tau^k, \lambda^k\right\}_{k=0}^{K-1}) := \frac{1}{2S} \sum_{i=1}^{S} \| x_i^{K-1} \left(\left\{W_t^k, W_e^k, \tau^k, \lambda^k\right\}_{k=0}^{K-1}; y_i \right) - \tilde{x}_i \|^2$$

- **Pro's**: flexible and powerful. It adapts to several other algorithms (primal-dual, ADMM...)
- **Con's**: the underlying problem changes at each iteration. No clear interpretation from an optimisation viewpoint!

[4] Monga, Li, Eldar, 2021

LISTA for image super-resolution

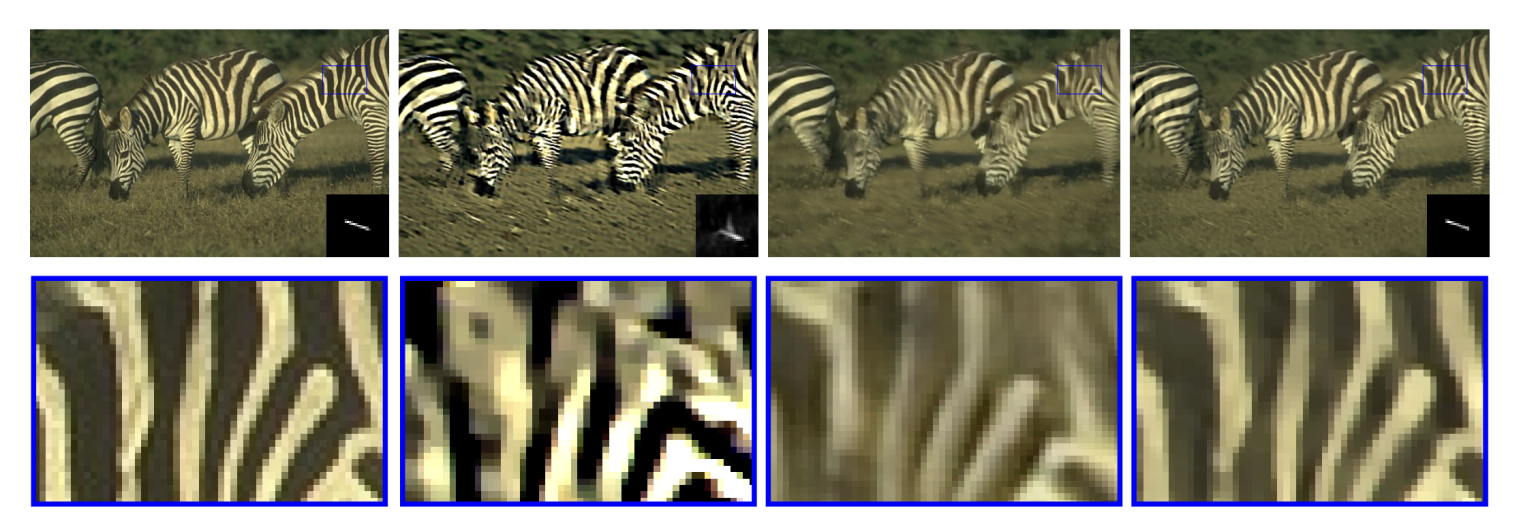


Figure 3: Left: GT, centre: standard approaches, Right: Unrolled version

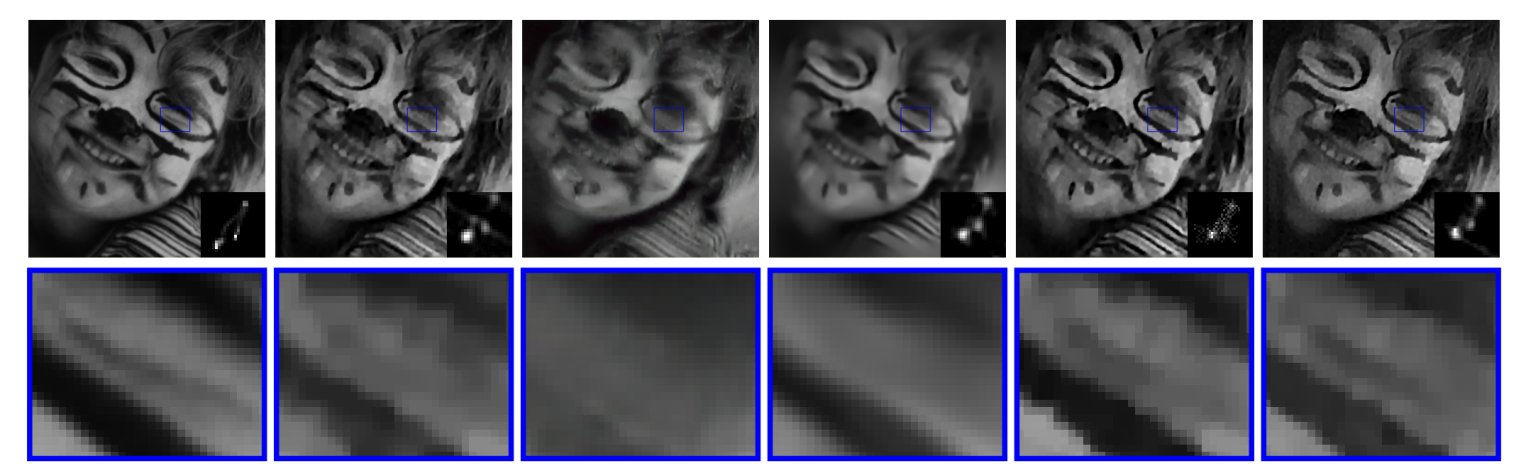


Figure 4: Left: GT, centre: standard approaches, Right: Unrolled version

Image denoisers

Image denoising

Goal: find a noise-free version $x \in \mathbb{R}^n$ of a noisy image $y \in \mathbb{R}^n$ s.t.:

$$y = x + b, \qquad b \sim \mathcal{N}(0, \sigma^2 \boldsymbol{Id})$$

where b is additive, white, Gaussian noise component.

Composite problem:

$$x^* \in \underset{x \in \mathbb{R}^n}{\arg\min} \underbrace{\frac{1}{2\sigma^2} \|x - y\|^2}_{f(x)} + g(x)$$

- Quadratic data term models Gaussian noise. It can be derived by MAP/maximum-likelihood estimation (see first lecture)
- $g(x)$ is the regularisation term enforcing prior information (sparsity, smoothness, gradient smoothness. . .)

Image denoising

Goal: find a noise-free version $x \in \mathbb{R}^n$ of a noisy image $y \in \mathbb{R}^n$ s.t.:

$$y = x + b, \qquad b \sim \mathcal{N}(0, \sigma^2 \boldsymbol{Id})$$

where b is additive, white, Gaussian noise component.

Composite problem:

$$x^* \in \operatorname*{arg\,min}_{x \in \mathbb{R}^n} \underbrace{\frac{1}{2\sigma^2}\|x - y\|^2}_{f(x)} + g(x)$$

- Quadratic data term models Gaussian noise. It can be derived by MAP/maximum-likelihood estimation (see first lecture)
- $g(x)$ is the regularisation term enforcing prior information (sparsity, smoothness, gradient smoothness. . .)

Observe that:

$$x^* \in \operatorname*{arg\,min}_{x \in \mathbb{R}^n} \frac{1}{2\sigma^2}\|x - y\|^2 + g(x) = \operatorname{prox}_{\sigma^2 g}(y)$$

Proximal operators as image denoisers

Goal: given $A \in \mathbb{R}^{m \times n}$, find $x \in \mathbb{R}^n$ from $y \in \mathbb{R}^m$ s.t.:

$$y = Ax + b, \qquad n \sim \mathcal{N}(0, \sigma^2 \boldsymbol{Id})$$

where b is additive, white, Gaussian noise component.

Consider now:

$$x^* \in \arg\min_{x \in \mathbb{R}^n} \frac{1}{2}\|Ax - y\|^2 + \lambda g(x)$$

Forward-backward scheme

x^0, $\tau \leq 1/L$, iterate for $k \geq 0$:

$$\begin{aligned} x^{k+1} &= \text{prox}_{\tau\lambda g}(x^k - \tau A^T(Ax^k - y)) \\ &= \arg\min_x \; g(x) + \frac{1}{2\tau\lambda}\|x - \left(x^k - \tau A^T(Ax^k - y)\right)\|^2 \approx \mathcal{D}(x^k - \tau A^T(Ax^k - y)) \end{aligned}$$

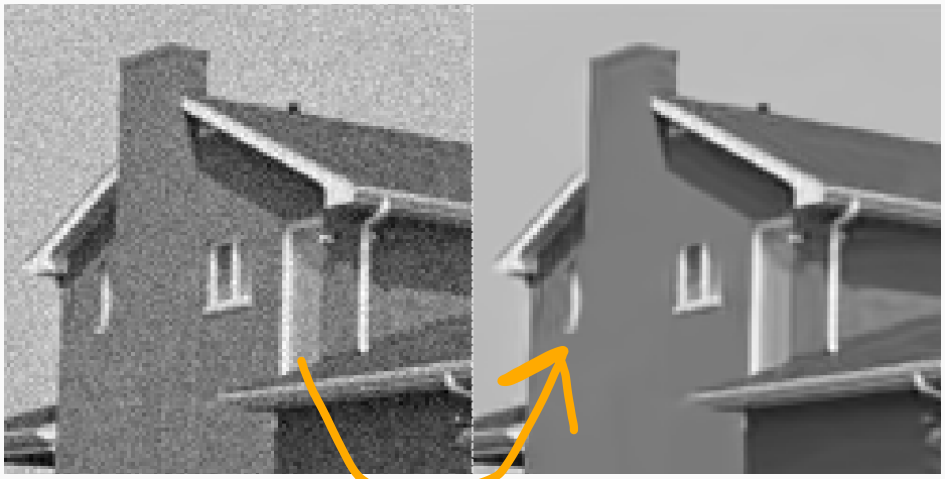
- Interpret proximal steps as **denoisers** $\mathcal{D}$ (i.e. operators) of gradient steps
- By definition of the proximal operator (i.e. appearance of $\|\cdot - \cdot\|^2$), assume **noise is Gaussian**

The regularisation function $g(x)$ is called *only* through its prox...

P&P idea

Plug & play methods: idea

Idea: replace $\text{prox}_{\tau\lambda g}$ by some off-the-shelf (black-box) denoiser $\mathcal{D} : \mathbb{R}^n \to \mathbb{R}^n$



Denoiser $\mathcal{D}$: anything doing this job (BM3D, your favourite PhotoShop function...)

Plug & play methods: idea

Idea: replace $\text{prox}_{\tau\lambda g}$ by some off-the-shelf (black-box) denoiser $\mathcal{D} : \mathbb{R}^n \to \mathbb{R}^n$

Denoiser $\mathcal{D}$: anything doing this job (BM3D, your favourite PhotoShop function...)

Plug & play forward-backward

x^0, $\tau \leq 1/L$, iterate for $k \geq 0$:

$$x^{k+1} = \mathcal{D}(x^k - \tau A^T(Ax^k - y))$$

- No need to define $g(x)$/tune λ! Any denoiser $\mathcal{D}$ (of additive, white Gaussian noise) would work
- $g(x)$ is implicitly defined by the action of the denoiser

Combining with DL

Idea: given training set $\{(\tilde{x}_i, y_i)\}_{i=1}^N$ of noise-free/noisy images, train a CNN denoiser $\mathcal{D} : \mathbb{R}^n \to \mathbb{R}^n$ to perform

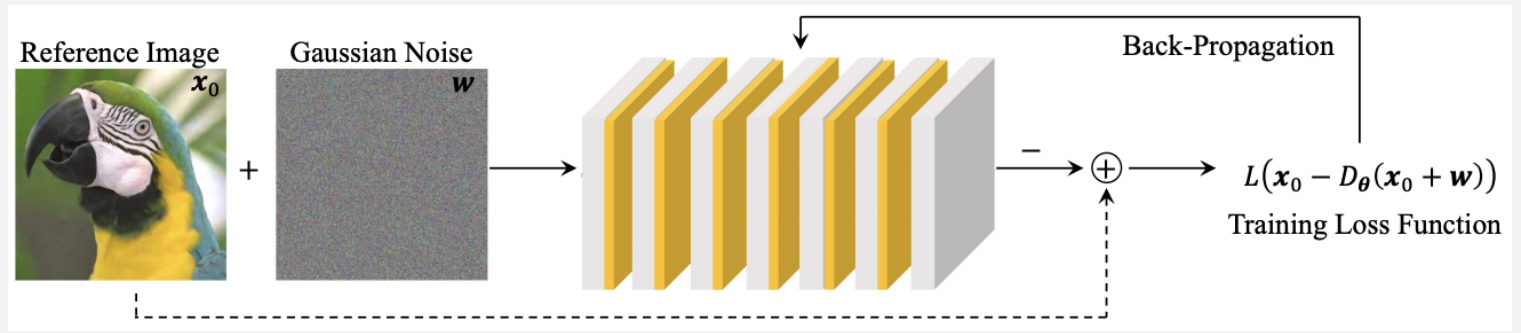
$$\mathcal{D}(y_i) = \tilde{x}_i, \quad \forall i = 1, \ldots, N$$

Integrating physical modelling in DL

Recall that A relates to the physical acquisition model (optical blur, downsampling).

$$x^{k+1} = \mathcal{D}(x^k - \tau A^T(Ax^k - y))$$

PnP approaches incorporate naturally **consistency** with the physics/acquired data.



Learning/using a CNN denoiser $\mathcal{D}_\theta$ on super-resolution problems, $A = SH \in \mathbb{R}^{m \times n}$.

Combining with DL

Idea: given training set $\{(\tilde{x}_i, y_i)\}_{i=1}^N$ of noise-free/noisy images, train a CNN denoiser $\mathcal{D} : \mathbb{R}^n \to \mathbb{R}^n$ to perform

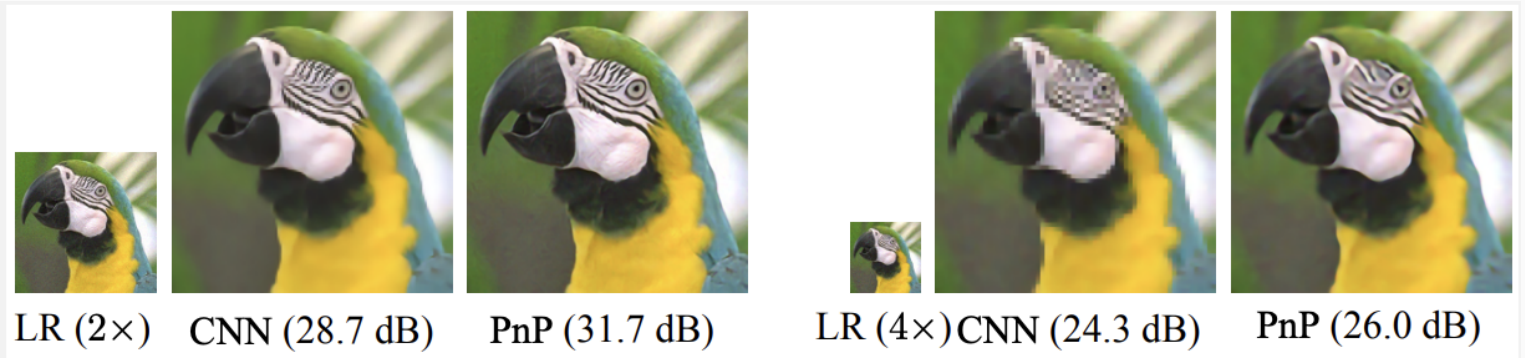
$$\mathcal{D}(y_i) = \tilde{x}_i, \quad \forall i = 1, \ldots, N$$

Integrating physical modelling in DL

Recall that A relates to the physical acquisition model (optical blur, downsampling).

$$x^{k+1} = \mathcal{D}(x^k - \tau A^T(Ax^k - y))$$

PnP approaches incorporate naturally **consistency** with the physics/acquired data.



Learning/using a CNN denoiser $\mathcal{D}_\theta$ on super-resolution problems, $A = SH \in \mathbb{R}^{m \times n}$.

regularization function

We thus have an interpretation:

composite optimisation/proximal algorithms $\Rightarrow$ denoisers $\mathcal{D}$

But the question is:

given a denoiser $\mathcal{D}$, is there a composite optimisation problem related to $\mathcal{D}$?

$$\exists \quad \arg\min_x \; F(x) := f(x) + g_{\mathcal{D}}(x) \quad ???$$

- Existence of $g_{\mathcal{D}}$? Convergence of $F(x)$?
- Given $\mathcal{D}$, is it the gradient/prox of some function?
- No convexity expected in general

PnP approaches with guarantees

Regularisation by denoising (RED)

Smooth case: given $x_0 \in \mathbb{R}^n$ and $\tau \leq 1/L$, iterate

$$x^{k+1} = x^k - \tau H(x^k), \qquad H(x^k) := \nabla f(x^k) + \lambda(x^k - \mathcal{D}(x^k))$$

where $\nabla f(x^k) = \nabla \left(\frac{1}{2}\|Ax - y\|^2\right) = A^T(Ax^k - y)$

Under conditions on $\mathcal{D}$ (symmetric Jacobian) $\lambda(x^k - \mathcal{D}(x^k))$ corresponds to the gradient of a function ∇g defined by[5]

$$g(x) = \frac{\lambda}{2} x^T (x - \mathcal{D}(x))$$

[5] Romano, Elad, Milanfar, '17

Proximal denoisers for convergent plug-and-play

Non-smooth case: for convergence guarantees, the denoiser should have the **gradient-step** structure:

$$\mathcal{D}_\varsigma(x) = \nabla\left(\frac{1}{2}\|x\|^2 - f_\varsigma(x)\right) = x - \nabla f_\varsigma(x) = (\boldsymbol{Id} - \nabla f_\varsigma)(x)$$

where $f_\varsigma : \mathbb{R}^n \to \mathbb{R}$ is a scalar function parametrised by a NN and $\mathcal{D}_\varsigma$ is trained to denoise images with Gaussian noise of level ς.

Theorem (proximal structure)

- If $\frac{1}{2}\|x\|^2 - f_\varsigma(x)$ is convex, then there exists $g_\varsigma : \mathbb{R}^n \to \mathbb{R} \cup +\{\infty\}$ s.t.:

$$\forall x \in \mathbb{R}^n, \quad \mathcal{D}_\sigma(x) \in \text{prox}_{g_\varsigma}(x) \qquad \text{(multi-valued)}$$

- If ∇f_ς is L-Lipschitz with $L < 1$, then $\mathcal{D}_\varsigma$ is injective and $\mathcal{D}_\varsigma(x) = \text{prox}_{g_\varsigma}(x)$, i.e. $\text{prox}_{g_\varsigma}$ is **single-valued**.

Note: g_ς is not necessarily convex, but its proximal operator is single-valued.

Convergence of PnP forward-backward

It is thus possible to define a non-convex g_ς such that $\text{prox}_{g_\varsigma} = \mathcal{D}_\varsigma$. For $\lambda > 0$ regularisation parameter, we now target the function:

$$F(x) := \lambda f(x) + g_\varsigma(x)$$

PnP forward-backward

$x^0 \in \mathbb{R}^n$, $\tau = 1$, minimise F by iterating for $k \geq 0$

$$z_{k+1} = x_k - \alpha \nabla f(x_k)$$
$$x_{k+1} = \mathcal{D}_\varsigma(z_{k+1}) \qquad \text{(PnP-proximal step)}$$

Theorem (convergence of PnP forward-backward)

If g_ς is C^2 with L-Lipschitz gradient with $L < 1$ and f is smooth with L_f Lipschitz gradient. Then, for $\alpha L_f < 1$, there holds:

- $F(x_k)$ is non-increasing and convergent
- All cluster points of (x_k) are stationary points of F
- Under suitable further assumptions on f and g_ς, the sequence (x_k) converges to a stationary point of F.

Unrolling + plug & play

A different approach to LISTA

$$x_{k+1} = \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y)) \longrightarrow \mathcal{D}_\theta(\underbrace{(\mathrm{Id} - \tau A^T A)}_{W_t} x_k + \underbrace{\tau A^T}_{W_e} y))$$

Idea: replace the soft-thresholding activation function by a parametrized (denoising) network and follow the idea of LISTA.

Learnable parameters become: $\{W_e, W_t, \theta\}$ [6]. The learned denoiser just acts **as implicit** (since not explicit expression) **regularisation functional** on the image.

[6] T. Meinhardt, M. Moeller, C. Hazirbas, and D. Cremers, '17

A different approach to LISTA

$$x_{k+1} = \mathcal{T}_{\tau\lambda}(x_k - \tau A^T(Ax_k - y)) \longrightarrow \mathcal{D}_\theta(\underbrace{(\mathrm{Id} - \tau A^T A)}_{W_t} x_k + \underbrace{\tau A^T}_{W_e} y))$$

Idea: replace the soft-thresholding activation function by a parametrized (denoising) network and follow the idea of LISTA.

Learnable parameters become: $\{W_e, W_t, \theta\}$ [6]. The learned denoiser just acts **as implicit** (since not explicit expression) **regularisation functional** on the image.

Questions:

- By learning all parameters, no clear understanding of which functional F is being minimized
- What parameters is better to learn?
- How to choose K?

[6] T. Meinhardt, M. Moeller, C. Hazirbas, and D. Cremers, '17

Bibliography

. . . very active research area! Contact me if interested!


S.V. Venkatakrishnan, C.A. Bouman, and B. Wohlberg, *Plug-and-Play Priors for Model Based Reconstruction*, GlobalSIP, 2013.


Romano, Y., Elad, M., and Milanfar, P., *The little engine that could: Regularization by denoising (RED)*, SIAM Journal on Imaging Sciences, 10(4):1804-1844, 2017.


Reehorst, E. T. and Schniter, P., *Regularization by denoising: Clarifications and new interpretations*, IEEE transactions on Computational Imaging, 5(1):52-67, 2018.


Kamilov, U. S., Bouman, C. A., Buzzart, G. T. and Wohlberg, B., *Plug-and-Play Methods for Integrating Physical and Learned Models in Computational Imaging*, IEEE Signal Processing Magazine, 40(1), 2023. Project page


Hurault, S., Leclaire, A. and Papadakis, N., *Proximal Denoiser for Convergent Plug-and-Play Optimization with Nonconvex Regularization*, International Conference on Machine Learning, 2022.

Questions?

calatroni@i3s.unice.fr